Human-in-the-Loop (HITL) (실습)

3-1. 왜 사람의 확인이 필요한가
3-2. interrupt로 실행 멈추기 (실습)
3-3. 승인, 수정, 거절 흐름 (실습)

3-1. 왜 사람의 확인이 필요한가

> 판단은 AI가 하고 최종 확정과 발송 승인은 사람이 한다는 원칙과 그 기반인 체크포인터를 이해해봅니다.

4일차에서 만든 Agent는 LLM이 도구 호출을 결정하면 즉시 실행합니다. 조회라면 문제가 없습니다. 출장 신청서 제출이나 출장 승인처럼 되돌리기 어려운 작업이라면 이야기가 다릅니다.

되돌리기 어려운 작업은 위험합니다. 사용자가 진짜 그것을 의도했는지 LLM이 잘못 이해했을 수 있습니다. 게다가 실행한 뒤에 되돌릴 절차도 없습니다.

그래서 원칙을 세웁니다. 판단은 AI가 하되 되돌리기 어려운 작업의 최종 확정과 발송 승인은 사람이 합니다. 모든 도구에 승인을 걸면 자동화의 의미가 없으므로 위험도별로 다르게 처리합니다.

도구 종류	위험도	처리
조회 (`search_employee` , `get_asset` , `list_projects`)	낮음	자동 실행
계산, 요약, 번역	낮음	자동 실행
출장 신청서 제출 (`request_business_trip`)	높음	사람 승인 필수
출장 승인 처리 (`approve_business_trip`)	매우 높음	사람 승인 필수

HITL(Human-in-the-loop)은 AI가 판단하고 사람이 중간에 확인해 승인하는 구조입니다. 구현하려면 기반 지식이 두 개 필요합니다. 그래프가 멈춘 시점의 State를 저장해 두는 checkpointer와 그 State를 저장할 때 쓰는 키인 thread_id입니다.

먼저 checkpointer입니다. 실행을 멈추다가 나중에 재개한다는 것은 곧 멈춘 시점의 상태를 어딘가에 저장해 둔다는 뜻입니다. LangGraph는 그래프의 매 단계마다 State 전체를 checkpointer에 저장합니다.

이번 과정은 SQLite 방식(`SqliteSaver`)을 표준으로 씁니다. 파일에 저장하므로 프로세스를 껐다가 다시 켜도 상태가 유지됩니다. 파일 이름은 `checkpoints.sqlite` 로 통일합니다.

개발 단계에서 잠깐 쓰는 `InMemorySaver`와는 인터페이스가 같습니다. 그래서 객체 한 줄만 바꾸면 서로 갈아 끼울 수 있습니다.

다음은 thread_id입니다. 대화 세션 ID라고 생각하면 됩니다. 같은 thread_id로 invoke하면 저장된 상태를 그대로 이어가고 다른 thread_id로 invoke하면 완전히 새 세션이 되어 서로 격리됩니다.

따라하기

1. checkpointer와 thread_id의 동작을 먼저 확인합니다. 멈추는 법을 배우기 전에 기억하는 법부터입니다. 아래 코드를 `day5_practice/checkpointer_basic.py` 로 저장해 실행합니다.

```
# checkpointer_basic.py - SQLite 체크포인터와 thread_id 기초
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_aws import ChatBedrockConverse
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_core.messages import HumanMessage

load_dotenv()

llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)

# SqliteSaver: 파일에 저장하므로 스크립트를 껐다가 다시 켜도 대화가 유지됩니다
def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


with SqliteSaver.from_conn_string("checkpoints.sqlite") as checkpointer:
    agent = create_agent(
        model=llm,
        tools=[],
        checkpointer=checkpointer,   # 여기에 붙입니다
    )

    # thread_id는 config로 전달합니다. 이 형태를 그대로 익혀 둡니다.
    config = {"configurable": {"thread_id": "user_A"}}

    r1 = agent.invoke({"messages": [HumanMessage("내 이름은 김하늘이야")]}, confi
g=config)
    print("1차:", get_text(r1["messages"][-1]))

    r2 = agent.invoke({"messages": [HumanMessage("내 이름이 뭐였지?")]}, config=
config)
    print("2차(같은 thread):", get_text(r2["messages"][-1]))

    config_b = {"configurable": {"thread_id": "user_B"}}
    r3 = agent.invoke({"messages": [HumanMessage("내 이름이 뭐였지?")]}, config=
config_b)
    print("3차(다른 thread):", get_text(r3["messages"][-1]))
```

2. 스크립트를 종료하고 다시 실행해서 같은 `thread_id` 로 "내 이름이 뭐였지?"만 물어봅니다. 파일 (`checkpoints.sqlite`)에 저장되어 있으므로 여전히 기억합니다.

   실행 결과 예시

   ```
   1차: 반갑습니다, 김하늘 님!
   2차(같은 thread): 김하늘 님이라고 하셨습니다.
   3차(다른 thread): 이전 대화 기록이 없어 알 수 없습니다.
   ```

3. 체크포인터가 보관하는 것은 마지막 상태 하나가 아닙니다. **단계마다 하나씩 쌓입니다.** 그래서 목록을 꺼내 특정 시점을 고르고 그 지점부터 다시 실행할 수 있습니다. 타임트래블(time travel)이라고 부릅니다.

   ```
   # 이 thread의 체크포인트 목록 (최신순)
   for snap in agent.get_state_history(config):
       print(snap.config["configurable"]["checkpoint_id"], len(snap.values["mes
   sages"]))

   # 돌아가고 싶은 시점의 config를 그대로 넘기면 그 상태에서 이어서 실행됩니다
   past = list(agent.get_state_history(config))[2]     # 2단계 전
   agent.invoke(None, config=past.config)

   # 그 시점의 State를 고쳐서 다시 실행할 수도 있습니다
   agent.update_state(past.config, {"messages": [HumanMessage("질문을 바꿉니다")]})
   ```

   이때 원래 이력이 사라지지 않고 그 지점에서 가지가 하나 더 생깁니다. git에서 과거 커밋으로 돌아가 브랜치를 띄우는 것과 같습니다. 필요 조건은 하나입니다. `thread_id` 가 같아야 합니다.

   다음에 나올 HITL이 정확히 이 구조 위에서 돕니다. 사람 승인을 기다리며 멈춰 있다가 재개하는 것도, 3-3의 수정 흐름처럼 인자를 고쳐서 실행하는 것도 전부 이 기능입니다.
   디버그할 때도 유용합니다. Agent가 이상한 답을 내면 처음부터 다시 돌리는 대신 어긋나기 시작한 지점으로만 되돌려 프롬프트를 바꿔 볼 수 있습니다.

막힐 때

- `SqliteSaver` import 오류가 납니다.

  langgraph-checkpoint-sqlite가 설치되어 있어야 합니다. 1일차 requirements.txt에 들어 있으니 레포 루트에서 `python -m pip install -r requirements.txt` 를 다시 돌려 봅니다.

- `Checkpointer requires one or more of the following 'configurable' keys: thread_id` 오류가 납니다.

  `config` 를 안 넘겨주었거나 `{"configurable": {...}}` 껍데기를 빠뜨린 것입니다. `config={"thread_id": ...}` 가 아니라 `config={"configurable": {"thread_id": ...}}` 입니다.

- `with` 블록 밖에서 agent를 쓰면 연결이 닫혀 오류가 납니다.

  실습 코드는 `with` 안에서 전부 끝냅니다.

- thread_id 설계 관례가 궁금합니다.

  멀티유저는 `thread_id = user_id` 로 만들고 한 사용자의 여러 대화방은 `thread_id = f"{user_id}:{conv_id}"` 로 만듭니다.

확인: 같은 thread_id는 이름을 기억하고 다른 thread_id는 모르며 재실행 후에도 기억이 유지되면 성공입니다.

> 실행을 멈추고 재개하는 기반이 체크포인터와 thread_id라는 걸 확인했습니다.

3-2. interrupt로 실행 멈추기 (실습)

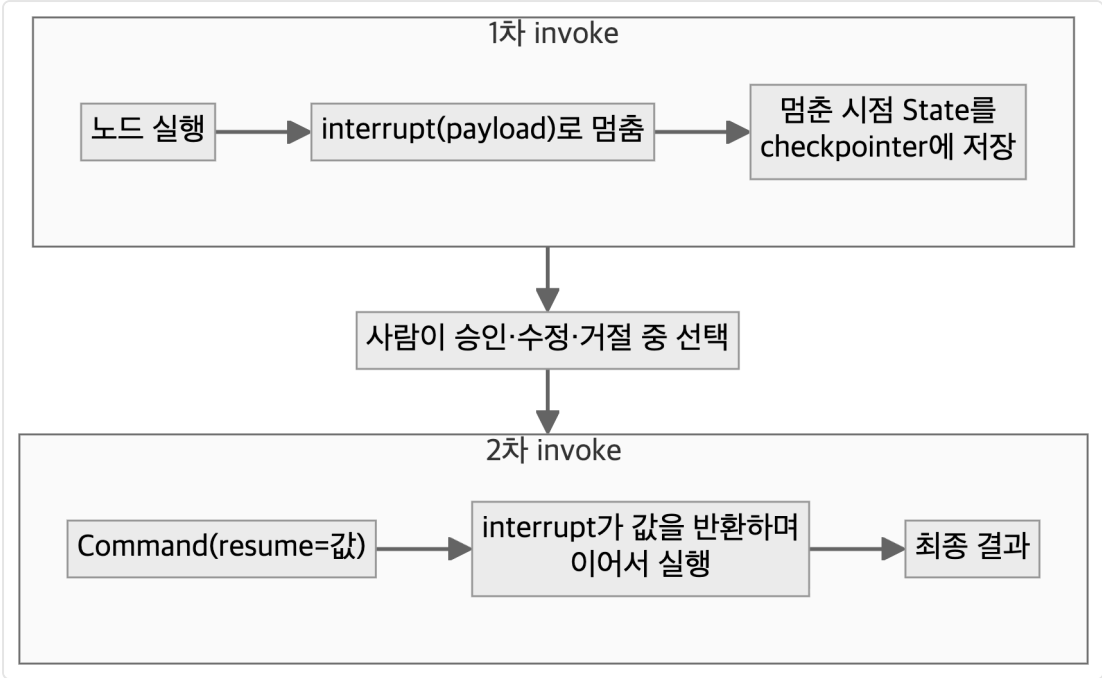
> 위험한 도구 호출 전에 실행을 멈추고 사람의 응답으로 재개하는 실행 모델을 익혀봅니다.

이 절이 오늘 과정에서 가장 낯선 구간입니다. 한 번의 실행이 두 번의 invoke로 쪼개지기 때문입니다.
`interrupt()` 의 동작을 4단계로 정리합니다.

1. 노드 안에서 `interrupt(payload)` 를 호출하면 그래프가 즉시 멈춥니다.
2. payload(어떤 승인을 요청하는지 담은 정보)는 호출자에게 전달됩니다.
3. 이 시점의 State는 checkpointer에 저장됩니다. 그래서 checkpointer가 필수입니다.
4. 나중에 `Command(resume=값)` 으로 다시 invoke하면 멈췄던 지점부터 재개되고 그 값이 `interrupt()` 의 반환값이 됩니다.

4번이 가장 낯섭니다. `decision = interrupt({...})` 라고 쓴 한 줄은 1차 invoke에서 그 자리에 멈춥니다. 그러다가 2차 invoke가 `Command(resume="yes")` 를 보내면 그 줄이 "yes"를 반환하며 이어집니다.

`decision` 변수에는 사람의 응답이 들어옵니다. 한 줄의 코드가 두 번의 invoke에 걸쳐 실행되는 셈입니다. 이 감각이 HITL의 전부입니다.


1차 invoke
노드 실행
interrupt(payload)로 멈춤
멈춘 시점 State를 checkpointer에 저장
사람이 승인·수정·거절 중 선택
2차 invoke
Command(resume=값)
interrupt가 값을 반환하며 이어서 실행
최종 결과

따라하기

1. 관찰이 쉽도록 그래프를 직접 조립한 최소 예제를 `day5_practice/interrupt_basic.py` 로 저장해 실행합니다.

```
# interrupt_basic.py - 그래프를 멈추고 사람을 기다리기 (체크포인터 필수)
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.types import interrupt, Command
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_core.messages import HumanMessage, AIMessage


class State(TypedDict):
    messages: Annotated[list, add_messages]
    pending_action: str
    decision: str


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


def propose_action(state: State) -> dict:
    """사용자가 위험 작업을 요청했다고 가정합니다."""
    return {"pending_action": "출장 신청서 제출: 부산 2일, 예산 800,000원"}


def human_approval(state: State) -> dict:
    """interrupt: 사람 승인을 기다립니다."""
    decision = interrupt({
        "type": "approval",
        "action": state["pending_action"],
        "question": "이 작업을 실행할까요? (yes/no)",
    })
    return {"decision": decision}


def execute(state: State) -> dict:
    if state["decision"] == "yes":
        return {"messages": [AIMessage(content=f"실행됨: {state['pending_actio
n']}")]}
    return {"messages": [AIMessage(content="실행 거부됨")]}
```

```
builder = StateGraph(State)
builder.add_node("propose", propose_action)
builder.add_node("approval", human_approval)
builder.add_node("execute", execute)
builder.add_edge(START, "propose")
builder.add_edge("propose", "approval")
builder.add_edge("approval", "execute")
builder.add_edge("execute", END)


def main():
    # 체크포인터 필수! 없으면 interrupt 시점에 오류가 납니다.
    with SqliteSaver.from_conn_string("checkpoints.sqlite") as checkpointer:
        graph = builder.compile(checkpointer=checkpointer)
        config = {"configurable": {"thread_id": "hitl_demo"}}

        # 1차 invoke: interrupt에서 멈춥니다
        print("[1차 invoke]")
        graph.invoke({"messages": [HumanMessage("작업 시작")]}, config=config)

        # 멈춘 상태 확인
        state = graph.get_state(config)
        print("  멈춘 노드:", state.next)                        # ('approva
l',)
        print("  요청 내용:", state.tasks[0].interrupts[0].value)

        # 사람에게 묻습니다 (실제 서비스에서는 웹 UI나 메신저 버튼)
        user_decision = input("\n승인하시겠습니까? (yes/no): ").strip()

        # 2차 invoke: Command로 재개합니다
        print("\n[2차 invoke]")
        result = graph.invoke(Command(resume=user_decision), config=config)
        print("  최종:", get_text(result["messages"][-1]))


if __name__ == "__main__":
    main()
```

실행 결과 예시

```
[1차 invoke]
  멈춘 노드: ('approval',)
  요청 내용: {'type': 'approval', 'action': '출장 신청서 제출: 부산 2일, 예산 800,000원',
              'question': '이 작업을 실행할까요? (yes/no)'}

승인하시겠습니까? (yes/no): yes

[2차 invoke]
  최종: 실행됨: 출장 신청서 제출: 부산 2일, 예산 800,000원
```

막힐 때

- checkpointer 없이 compile하면 interrupt 시점에 오류가 납니다.

  멈춘 상태를 저장할 곳이 없기 때문입니다. `compile(checkpointer=...)` 를 확인합니다.

- 2차 invoke에서 thread_id가 다르면 멈춘 지점을 못 찾고 새 실행이 시작됩니다.

  1차와 2차의 `config` 가 반드시 같아야 합니다. 가장 흔한 실수입니다.

- `Command(resume=...)` 대신 새 메시지를 invoke하면 재개가 아니라 이상한 동작이 됩니다.

  재개는 반드시 `Command` 로 합니다.

- 1차 invoke의 반환값에 최종 답이 없습니다.

  정상입니다. 멈추기 때문입니다. 멈췄는지는 `graph.get_state(config).next` 가 비어 있지 않은 것으로 확인합니다.

- interrupt가 있는 노드는 재개 시 노드 처음부터 다시 실행됩니다.

  interrupt 앞에 비싼 작업(API 호출 등)을 두지 말고 interrupt를 노드 앞부분에 둡니다.

- 같은 thread_id로 반복 실행하면 이전 실행의 상태가 남아 헷갈립니다.

  실습 중에는 실행마다 thread_id를 바꾸는 것이 안전합니다.

확인: 1차 invoke가 approval 노드에서 멈추고 yes 입력 후 2차 invoke에서 실행됨 메시지가 나오면 성공입니다.

> 한 줄의 interrupt 코드가 두 번의 invoke에 걸쳐 실행되는 HITL 실행 모델을 경험했습니다.

3-3. 승인, 수정, 거절 흐름 (실습)

> resume 값에 dict를 실어 승인, 수정, 거절 세 갈래를 처리해봅니다.

실제 HITL은 yes/no 두 갈래가 아니라 사실 세 갈래입니다.

선택	의미	처리
승인 (approve)	제안된 그대로 실행	도구를 원래 인자로 실행
수정 (modify)	인자를 고쳐서 실행	사람이 준 인자로 바꿔서 실행
거절 (reject)	실행하지 않음	거절 사실을 Agent에 알려 대안을 제시하게 함

`Command(resume=값)`에는 어떤 파이썬 객체든 넣을 수 있습니다. 문자열 "approve"만 보내는 것이 아니라 `{"decision": "modify", "args": {...}}` 같은 dict를 보내면 수정 흐름이 됩니다.

시나리오는 출장 업무입니다. 위험 도구는 출장 신청서를 제출하는 `request_business_trip`과 출장을 승인하는 `approve_business_trip` 두 개입니다.

LLM이 이 도구를 호출하기로 결정하면 실행 직전에 멈춥니다. 사람은 승인, 수정, 거절 중 하나로 응답합니다.

조회 도구는 멈추지 않고 자동 실행합니다.

따라하기

1. 승인, 수정, 거절을 모두 처리하는 그래프를 작성해 `day5_practice/hitl_flow.py` 로 저장하고 실행합니다.

```
# hitl_flow.py - HITL 3갈래: 승인 / 수정 / 거절
from typing import TypedDict, Annotated
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.messages import HumanMessage, ToolMessage
from langchain_core.tools import tool
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.types import interrupt, Command
from langgraph.checkpoint.sqlite import SqliteSaver

load_dotenv()


@tool
def request_business_trip(destination: str, days: int, budget: int) -> str:
    """출장 신청서를 제출한다. 되돌리기 어려운 작업이므로 사람 승인이 필요하다."""
    return f"출장 신청서 제출 완료: {destination} {days}일, 예산 {budget:,}원 (교육용 더미 처리)"


@tool
def approve_business_trip(trip_id: str) -> str:
    """접수된 출장 신청을 승인 처리한다. 되돌리기 어려운 작업이므로 사람 승인이 필요하다."""
    return f"출장 신청 {trip_id} 승인 완료 (교육용 더미 처리)"


@tool
def search_trip_policy(query: str) -> str:
    """사내 출장 규정을 검색한다. 조회 전용이라 승인이 필요 없다."""
    return "출장 규정: 국내 출장 예산 한도는 1일 400,000원입니다. (교육용 더미 데이터)"


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinstance(block, dict))
    return content


RISKY_TOOLS = {"request_business_trip", "approve_business_trip"}
TOOLS = [request_business_trip, approve_business_trip, search_trip_policy]
TOOL_MAP = {t.name: t for t in TOOLS}
```

```
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)
llm_with_tools = llm.bind_tools(TOOLS)


class State(TypedDict):
    messages: Annotated[list, add_messages]


def agent(state: State) -> dict:
    """LLM이 도구 호출을 결정하는 노드"""
    return {"messages": [llm_with_tools.invoke(state["messages"])]}


def human_gate(state: State) -> dict:
    """위험 도구 호출 전에 멈춰 사람의 결정을 받는 노드 (단순화: 위험 도구는 한 번에 하나)"""
    call = state["messages"][-1].tool_calls[0]
    answer = interrupt({
        "tool": call["name"],
        "args": call["args"],
        "question": "승인(approve), 수정(modify), 거절(reject) 중 선택하세요.",
    })
    # answer 예시:
    #   {"decision": "approve"}
    #   {"decision": "modify", "args": {"budget": 500000}}
    #   {"decision": "reject", "reason": "예산 초과"}
    decision = answer.get("decision", "reject")

    if decision == "reject":
        return {"messages": [ToolMessage(
            content=f"사용자가 실행을 거절했습니다. 사유: {answer.get('reason', '없
음')}. "
                    "실행하지 말고 대안을 안내하세요.",
            tool_call_id=call["id"],
        )]}

    args = call["args"]
    if decision == "modify":
        # 사람이 준 인자로 덮어씁니다 (부분 수정 허용)
        args = {**args, **answer.get("args", {})}

    result = TOOL_MAP[call["name"]].invoke(args)
    return {"messages": [ToolMessage(content=result, tool_call_id=call["i
d"])]}
```

```
def run_tools(state: State) -> dict:
    """안전한 도구는 승인 없이 실행하는 노드"""
    results = []
    for call in state["messages"][-1].tool_calls:
        output = TOOL_MAP[call["name"]].invoke(call["args"])
        results.append(ToolMessage(content=output, tool_call_id=call["id"]))
    return {"messages": results}


def route(state: State) -> str:
    last = state["messages"][-1]
    if not getattr(last, "tool_calls", None):
        return "end"
    if any(c["name"] in RISKY_TOOLS for c in last.tool_calls):
        return "gate"
    return "tools"


builder = StateGraph(State)
builder.add_node("agent", agent)
builder.add_node("gate", human_gate)
builder.add_node("tools", run_tools)
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", route, {"gate": "gate", "tools": "too
ls", "end": END})
builder.add_edge("gate", "agent")
builder.add_edge("tools", "agent")


def run_scenario(title: str, resume_value: dict):
    """1차 invoke로 멈추고 주어진 응답으로 재개하는 헬퍼"""
    print(f"\n=== {title} ===")
    with SqliteSaver.from_conn_string("checkpoints.sqlite") as checkpointer:
        graph = builder.compile(checkpointer=checkpointer)
        config = {"configurable": {"thread_id": f"trip_{title}"}}

        graph.invoke({"messages": [HumanMessage(
            "부산 2일 출장을 예산 80만 원으로 신청해줘"
        )]}, config=config)

        state = graph.get_state(config)
        print("  승인 요청:", state.tasks[0].interrupts[0].value["args"])

        result = graph.invoke(Command(resume=resume_value), config=config)
        print("  결과:", get_text(result["messages"][-1]))


if __name__ == "__main__":
```

```
    # 시나리오 1: 승인 – 제안 그대로 실행
    run_scenario("approve", {"decision": "approve"})

    # 시나리오 2: 수정 – 예산을 50만 원으로 고쳐서 실행
    run_scenario("modify", {"decision": "modify", "args": {"budget": 50000
0}})

    # 시나리오 3: 거절 – 실행하지 않고 대안 안내
    run_scenario("reject", {"decision": "reject", "reason": "예산 초과"})
```

2. 출장 규정을 묻는 질문("출장 예산 한도가 얼마야?")으로도 실행해 조회 도구는 멈추지 않고 바로 답이 나오는 것을 확인합니다.

실행 결과 예시

```
=== approve ===
  승인 요청: {'destination': '부산', 'days': 2, 'budget': 800000}
  결과: 출장 신청서가 제출되었습니다. 부산 2일, 예산 800,000원입니다.

=== modify ===
  승인 요청: {'destination': '부산', 'days': 2, 'budget': 800000}
  결과: 출장 신청서가 제출되었습니다. 부산 2일, 예산 500,000원입니다.

=== reject ===
  승인 요청: {'destination': '부산', 'days': 2, 'budget': 800000}
  결과: 요청하신 출장 신청은 진행하지 않았습니다. 예산을 조정해 다시 요청해 주세요.
```

위 코드의 거절은 거절 사실을 `ToolMessage`로 Agent에 돌려주는 방식입니다. LLM이 거절 사유를 보고 대안을 제시하므로 대화가 자연스럽게 이어집니다. 단순히 종료하고 싶으면 거절 시 고정 메시지를 반환하고 END로 보내는 분기를 추가하면 됩니다.

참고로 `create_agent`를 쓸 때는 이 게이트 로직을 직접 만들 필요가 없습니다. `HumanInTheLoopMiddleware(interrupt_on={"request_business_trip": True, "approve_business_trip": True})` 한 줄로 같은 효과를 냅니다. 5번째 세션 5-2에서 그렇게 씁니다.

다만 내부에서 무슨 일이 일어나는지는 지금 만든 그래프가 그대로 담고 있습니다.

막힐 때

- `answer.get(...)`에서 `AttributeError`가 납니다.

  `Command(resume=...)`에 문자열을 보냈는데 코드가 dict를 기대하는 경우입니다. interrupt를 설계할 때 resume 값의 형식을 먼저 정하고 보내는 쪽과 받는 쪽을 맞춥니다.

- 수정 흐름에서 사람이 안 보낸 인자가 사라집니다.

  `answer["args"]`로 전체를 덮어쓴 경우입니다. `{**원래args, **수정args}` 병합 패턴을 씁니다.

- 같은 thread_id로 시나리오를 반복 실행하면 이전 상태가 남아 헷갈립니다.

  위 코드처럼 시나리오마다 thread_id를 다르게 줍니다.

확인: 세 시나리오에서 각각 원래 인자로 실행, 500,000원으로 수정 실행, 거절 후 대안 안내가 나오면 성공입니다.

> resume 값에 구조화된 dict를 실어 승인, 수정, 거절 세 갈래를 처리하는 패턴을 익혔습니다.
